LOW LEVEL DESIGN DOCUMENT

EMPLOYEE MANAGEMENT SYSTEM

-Rajaram Balakrishnan

1. Introduction

1.1 Purpose

This document describes the low-level design of the Employee Management System implemented in C++. The system is designed to manage employee records efficiently by supporting operations such as adding, removing, searching, and updating employee information. The design is based on OOPS principles.

---

2. System Overview

2.1 System Description

The Employee Management System is implemented as a Menu-based application.

All employee types are managed through a common interface, allowing the system to handle them polymorphically. The application supports multiple menus that routes the user through different operations such as adding employees, retrieving details, and performing administrative tasks.

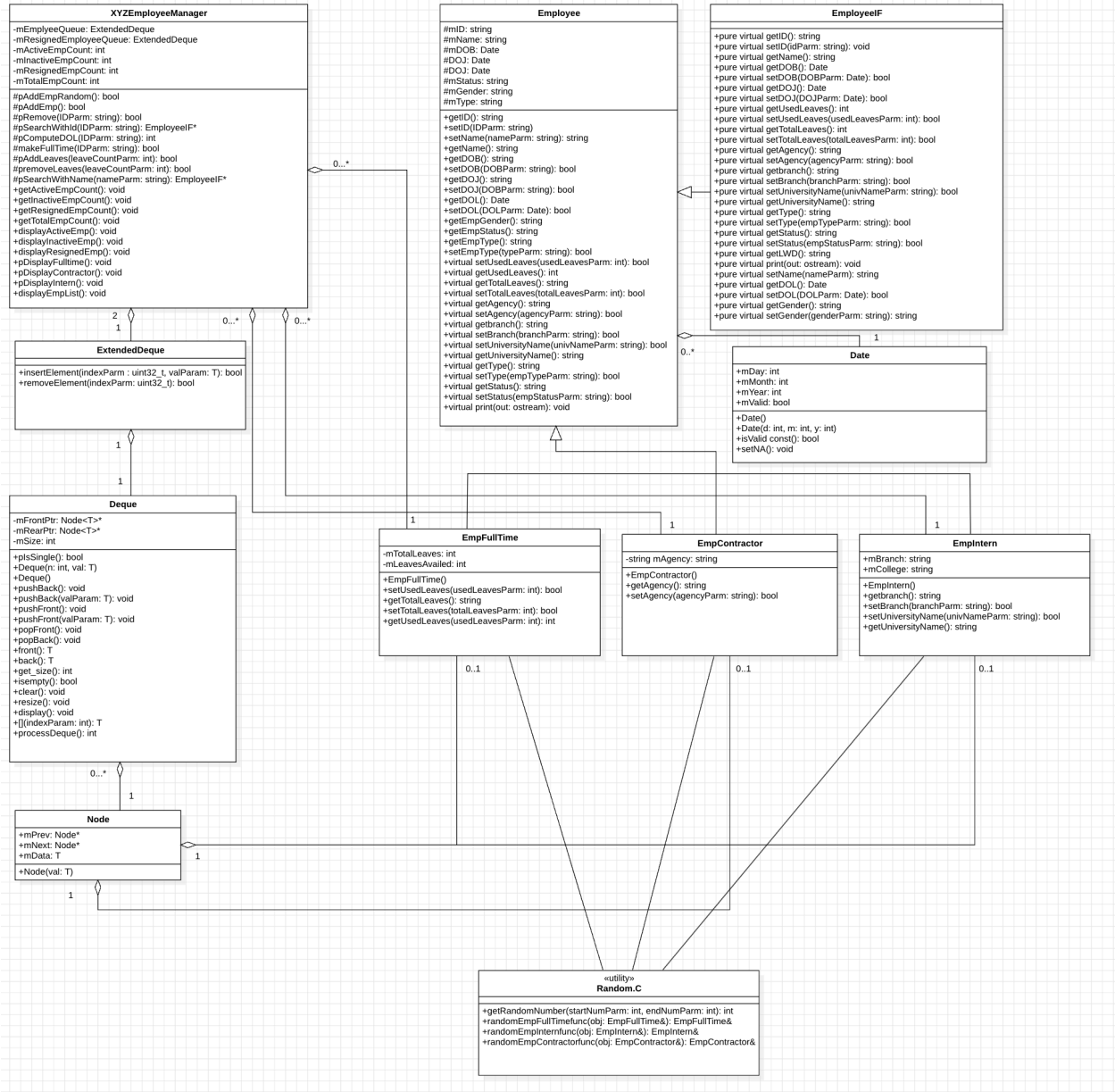
2.2 Key Features

The system supports creation of multiple employee types and provides flexible filtering capabilities based on employee type, gender, and status etc. It allows searching employees using unique identifiers or names and supports lifecycle management, including resignation handling and fulltime conversion.

2.3 Assumptions

Each employee is uniquely identified by an ID. All inputs are assumed to be provided via the console and are handled synchronously.

2.5 UML Class Diagram



3. Architecture Design

3.1 Overall Design

The system follows an object-oriented design. The architecture consists of three main layers:

- The user interface layer handles menu navigation and user interaction.
- The manager class (`XYZEmployeeManager`) performs all core operations using its methods.
- The data layer uses a custom deque to store employee objects.

3.2 Component Responsibilities

The menu functions are responsible for capturing user input and invoking appropriate operations in the manager class.

The data layer is implemented using a custom doubly linked list (`Deque` and `ExtendedDeque`) that provides efficient insertion and deletion operations.

3.3 Data Flow

User input is received through menu functions and passed to the manager class. The manager class processes the request, interacts with the underlying data structures, and returns the result, which is then displayed to the user.

---

4. Class Design

4.1 Design Overview

The system is built using an interface-based design combined with inheritance. This ensures that all employee types follow a common structure while allowing flexibility for type-specific behavior.

4.2 EmployeeIF (Interface)

The `EmployeeIF` interface defines the operations that any employee must support like methods for accessing and modifying attributes such as ID, name, dates, type, status, and other properties.

It also defines virtual methods for type-specific attributes such as leaves, agency, and university name so that classes can override only the neccessary methods.

4.3 Employee (Base Class)

The `Employee` class provides a concrete implementation of common attributes shared across all employee types. These include identification details, personal information, and employment-related data.

The class also provides default implementations for virtual functions, allowing derived classes to override only those behaviors which are specific to them.

4.4 Derived Classes

The system includes special classes for different employee types.

The `EmpFullTime` class extends the base class by adding attributes & methods related to leave management. It provides implementations for managing total and used leaves and represents permanent employees in the system.

The `EmpIntern` class adds an attribute to store the university and branch name. It represents temporary employees and supports conversion to full-time employees.

The `EmpContractor` class includes an agency attribute and represents employees associated with external agencies. It does not support leave management.

4.5 Manager Class (XYZEmployeeManager)

The `XYZEmployeeManager` class acts as the main controller of the EMS system. It maintains two primary Queues: one for current employees and another for resigned employees.

It provides functions for adding employees, removing employees, searching by ID or name, updating leaves, converting employee types, and displaying employee data. It also maintains counters to track active, inactive, and resigned employees.

---

5. Detailed methods Logic

5.1 Add Employee

The system allows adding employees either randomly or based on user selection of type. Depending on the input, the corresponding derived class object is created and inserted into the active employee queue.

5.2 Remove Employee

When an employee is removed, the system identifies the employee using the provided ID, updates the status to resigned, and moves the object to the resigned employee queue.

5.3 Search Operations

Search operations are implemented using linear traversal. The system supports searching by ID and name. If a match is found, the corresponding object is returned.

5.4 Display Operations

The system supports displaying employees based on different filters such as type, gender, and status. This is achieved by iterating through the employee collection and applying conditional checks.

5.5 Leave Management

Leave updates are applied only to full-time employees. The system iterates through all employees and updates leave balances for those matching the required type.

5.6 Intern(or Contractor) to Full-Time Conversion

The system supports converting an intern into a full-time employee by creating a new object, copying relevant data, and replacing the existing object in the collection.

---

6. API Definition – EmployeeIF Interface

6.1 Overview

The `EmployeeIF` interface defines a common methods for all employee types in the system. It provides set of APIs for accessing and modifying employee-related attributes .

All derived classes such as Full-Time, Intern, and Contractor must implement these APIs.

6.2 Public API Details (Interface Methods)

6.2.1 Identification APIs

Function: `getID()`
Description: Retrieves the unique identifier of the employee.
Input: None
Output: `std::string` – Employee ID

Function: `setID(std::string idParm)`
Description: Sets the unique ID for the employee.
Input: `idParm` – Employee ID
Output: `void`

6.2.2 Name APIs

Function: `getName()`
Description: Retrieves the name of the employee.
Input: None
Output: `std::string`

Function: `setName(std::string nameParm)`
Description: Sets the name of the employee.
Input: `nameParm` – Employee name
Output: `void`

6.2.3 Date APIs

Function: `getDOB()`
Description: Retrieves the date of birth of the employee.
Input: None
Output: `Date`

Function: `setDOB(Date DOBParm)`
Description: Sets the date of birth of the employee after validation.
Input: `DOBParm` – Date of Birth
Output: `bool` – Success/Failure

Function: `getDOJ()`
Description: Retrieves the date of joining.
Input: None
Output: `Date`

Function: `setDOJ(Date DOJParm)`
Description: Sets the date of joining.

Input: `DOJParm` – Date of Joining
Output: `bool`

Function: `getDOL()`
Description: Retrieves the date of leaving (if applicable).
Input: None
Output: `Date`

Function: `setDOL(Date DOLParm)`
Description: Sets the date of leaving.
Input: `DOLParm` – Date of Leaving
Output: `bool`

6.2.4 Leave Management APIs (Applicable to Full-Time Employees)

Function: `getUsedLeaves()`
Description: Retrieves the number of leaves used by the employee.
Input: None
Output: `int`

Function: `setUsedLeaves(int usedLeavesParm)`
Description: Sets the number of used leaves.
Input: `usedLeavesParm` – Number of leaves used
Output: `bool`

Function: `getTotalLeaves()`
Description: Retrieves the total number of leaves allocated.
Input: None
Output: `int`

Function: `setTotalLeaves(int totalLeavesParm)`
Description: Sets the total number of leaves.
Input: `totalLeavesParm` – Total leaves
Output: `bool`

6.2.5 Contractor-Specific APIs

Function: `getAgency()`
Description: Retrieves the agency associated with the contractor.
Input: None
Output: `std::string`

Function: `setAgency(std::string agencyParm)`
Description: Sets the contractor's agency name.
Input: `agencyParm` – Agency name
Output: `bool`

6.2.7 Intern-Specific APIs

Function: `getBranch()`
Description: Retrieves the branch or department of the employee.
Input: None
Output: `std::string`

Function: `setBranch(std::string branchParm)`
Description: Sets the branch of the employee.
Input: `branchParm` – Branch name
Output: `bool`

Function: `setUniversityName(std::string univNameParm)`
Description: Sets the university name for an intern.
Input: `univNameParm` – University name
Output: `bool`

Function: `getUniversityName()`
Description: Retrieves the university name.
Input: None
Output: `std::string`

6.2.8 Type and Status APIs

Function: `getType()`
Description: Retrieves the type of employee (Full-Time / Contractor / Intern).
Input: None
Output: `std::string`

Function: `setType(std::string empTypeParm)`
Description: Sets the employee type.
Input: `empTypeParm` – Type identifier
Output: `bool`

Function: `getStatus()`
Description: Retrieves the current status (Active / Inactive / Resigned).
Input: None
Output: `std::string`

Function: `setStatus(std::string empStatusParm)`
Description: Sets the employee status.
Input: `empStatusParm` – Status value
Output: `bool`

6.2.9 Gender APIs

Function: `getGender()`
Description: Retrieves the gender of the employee.
Input: None
Output: `std::string`

Function: `setGender(std::string genderParm)`
Description: Sets the gender of the employee.
Input: `genderParm` – Gender value
Output: `bool`

6.2.10 Utility APIs

Function: `print(std::ostream& out) const`
Description: Prints formatted employee details to the output stream.
Input: Output stream reference
Output: `void`